

WGPS MESSAGE COMMUNICATION FORMAT**(Wagon GPS Project of IR)**

(Annexure 1 has description of parameters/abbreviations used, Annexure 2 has the pre-defined values of these parameters)

As per oneM2M standards (www.onem2m.org)

Document applies for MQTT protocol with QoS 1 and JSON payload format

INDEX

Clause No.	Description	Page No.
1.0	Actionables for Vendor	2
2.0	Certificate generation and signing from field device of vendor	3
3.0	Resource Tree Structure	4
4.0	Containers Details	4-5
5.0	Device Topics	6
6.0	CRIS Topics	6
7.0	Various operations performed by the devices	6
8.0	FlexContainer Payloads with Data Types and Sample Values	7 - 11
9.0	Notification Payload	12
10.0	Firmware Update Procedure	13 - 15
11.0	Error Messages and Handling	16
12.0	Annexure-1: Parameter Description with Abbreviations Used	17 - 19
13.0	Annexure-2: Pre-defined Values of Parameters	20
14.0	Annexure-3: Generic Steps for creating a Certificate using openSSL	20- 23
15.0	Annexure-4: Variables used in the JSON payloads	23 - 24
16.0	Annexure-5: Registration of Vendors through Application	25
17.0	Contact Details	26

1.0 Actionables for Vendor who has been awarded the letter of acceptance:

- 1.1 Vendor shall acquire basic understanding of oneM2M messages. (Refer <https://ccspweb.railsaver.gov.in/docs/oneM2MGuideline.html>)
- 1.2 Vendor shall share its details as per Annexure 5 to CRIS.
- 1.3 Unique Vendor Code along with login credentials of WGPS mobile app will be shared by CRIS on email.
- 1.4 Before field installation of the devices, Vendor should register devices through WGPS Mobile App*. Vendor shall receive a configuration file in Mobile App after successful registration. The configuration file will consist of details (viz. url, registrar CSE (Common Service Entity), AEID, subjectAlt Name etc.).
- 1.5 Vendor will be provided Login details for CCSP Portal by CCSP Admin at email address provided by the vendor.
- 1.6 Vendor shall generate certificates for each of its devices and get it signed by the CA server of CRIS as detailed in para 2.

** This will create Application Entity resource <AE> registration and other resources for devices at oneM2M Common Service Platform. This platform is an IoT/M2M platform based on oneM2M standards*

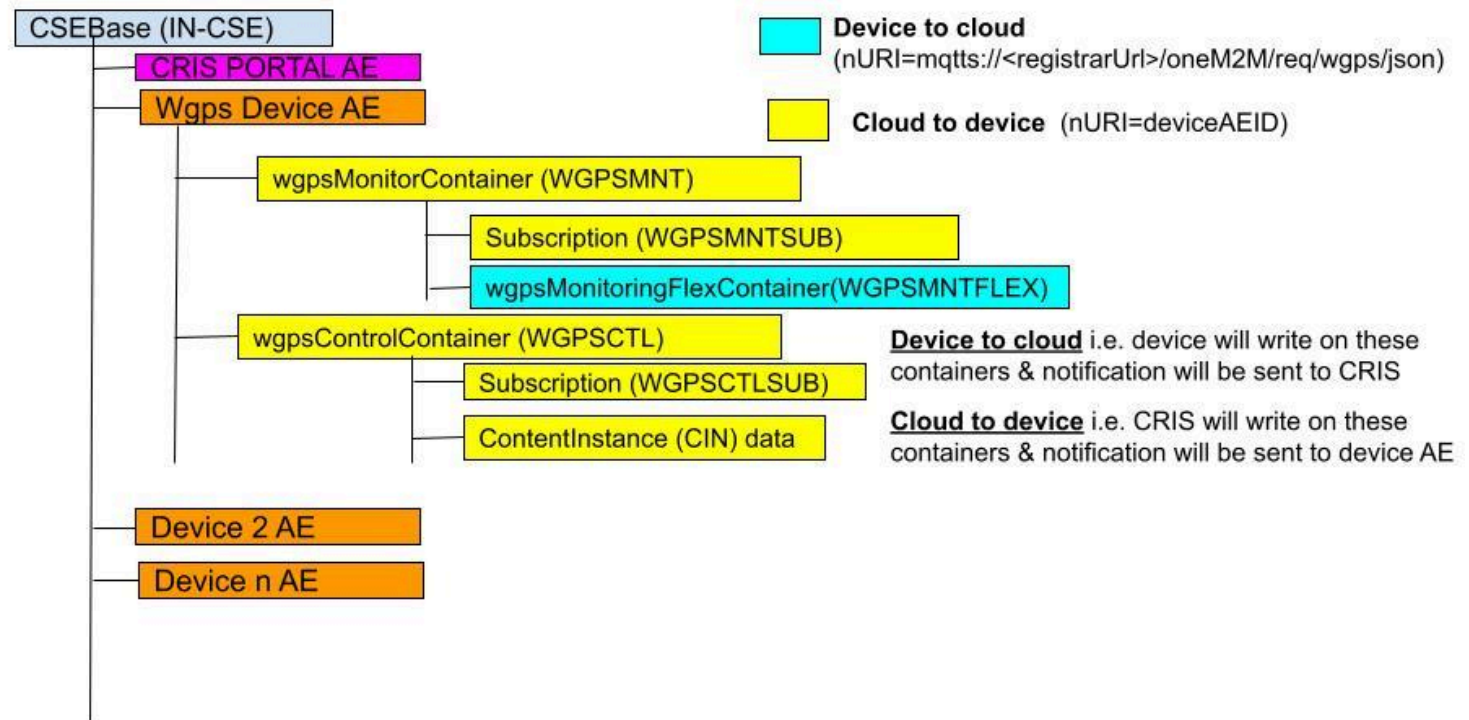
2.0 Certificate generation and signing from field device of vendor

2.1 The device vendor will have to create device specific certificates for communication of the device with CCSP over Mca (M2M AE and CSE communication) interface i.e for sending the monitoring data (i.e. creation of <contentInstance>, alerts and for receiving actuation, provisioning requests etc.

2.2. Following steps are required for generation of certificates for each device:

- 2.2.1 Download the root Certification Authority (CA) certificate using the login information provided for CCSP portal (Service Subscription Management (SSM) Module → Click on Certificate Management → Download root CA → click on download button to download root CA certificate.).
- 2.2.2 Copy the subject alt name from the configuration file shared by CRIS and generate device certificates using any popular library such as openssl, easyRSA etc. For procedures specific to each library refer Annexure 3 certificate generation using openssl and easyRSA.
- 2.2.3 Each device certificate shall be signed by CCSP CA. For more details refer Annexure 3.

3.0 Resource Tree Structure



4.0 Containers Details

All the following containers and subscriptions in these containers will be created by Vendor

Container Types	Container Resource Name	Subscription Resource Name	Used for	Direction of flow of information	Notification URI in Subscription Resource
wgpsMonitoringFlexContainer	WGPSMNT	WGPSMNTSUB	For storing GPS monitoring data from wgps tracker	device to cloud	nURI = CRIS nURI
wgpsControlFlexContainer	WGPSCTL	WGPSCTLSUB	For storing control commands from CRIS	cloud to device	nURI = device nURI

where

CRIS nURI=CRIS POA+topic=<registrarUrl>/oneM2M/req/wgps/json

All gateways/ devices while creating subscription resources for their container and sub container will put the nURI as mentioned in the table above. These topics will be subscribed by CRIS and CCSP will publish on this topic .

5.0 Device Topics:

Device will publish on :

Request Topic: /oneM2M/req/<DEVICE-AE-ID>/<registrarCseld>/json

Device will subscribe to :

Notification topic: /oneM2M/req/<registrarCseld>/<DEVICE-AE-ID>/json

6.0 CRIS Topics:

CRIS will subscribe to:

Notification topic: /oneM2M/req/wgps/json

7.0 Various operations performed by the devices

S.No.	Operation Type	Operation Description
1	Monitoring	Monitoring data of WGPS shall be published by the devices
2	Control	Control operation (Tracking Periodicity (Running), Tracking Periodicity (Static), Accelerometer Status, Permanent Device deactivation, Strategic Device deactivation, Temporary Device deactivation, Bluetooth Status, Device Reset)

8.0 Flex Container Payloads with Data Types and Sample Values:

CREATE Wagon Monitoring	CREATE Wagon Monitoring Sample Payload
<pre> { "op":1, "to":"<CONTAINER-RESOURCE-ID>", "fr":"<DEVICE -AE-ID>", "rqi":"<unique-Request-Identifier>", "rvi":"3", "ty":28, "pc":{ "hd:wgps": { "cnd":<String value>, "fw":<String value>, "pkSt":<String value>, "dgt":<String value>, "lat":<String value>, "latD":<String value>, "lng":<String value>, "lngD":<String value>, "spd":<Decimal value>, "noS":<Decimal value>, "hdop":<Decimal value>, "pdop":<Decimal value>, </pre>	<pre> { "op":1, "to":"R2279", "fr":"Sweb100", "rqi":"Get", "rvi":"3", "ty":28, "pc":{ "hd:wgps": { "cnd":"org.oneM2M.home.devices.wagonGps", "fw":"ABCD1234", "pkSt":"L", "dgt":"2023-12-12:10:15:34", "lat":"00.000000", "latD":"N", "lng":"00.000000", "lngD":"E", "spd":100.00, "noS":34, "hdop":1, "pdop":1, </pre>

<pre>"alt":<Decimal value>, "mcc":<String value>, "mnc":<String value>, "lac":<String value>, "cellId":<String value>, "rsi":<Decimal value>, "ber":<Decimal value>, "btPer":<Decimal value>, "mvnt":<Decimal value>, "tmp":<Decimal value>, "dst":<String value>, "cnt":<Decimal value>, "bleV":<Decimal value>, "bleL":<Decimal value>, "bleS":<Decimal value>, "recF":<Decimal value>, "tftA":<Decimal value>, "tnpA":<Decimal value>, "dacT":<Decimal value> } } }</pre>	<pre>"alt":4000, "mcc": "1234", "mnc": "1234", "lac": "1234", "cellId": "1234", "rsi":31, "ber":7, "btPer":100, "mvnt":1, "tmp":55.55, "dst": "2023-12-12:10:15:34", "cnt":100, "bleV":34, "bleL":55, "bleS":1, "recF":1, "tftA":1, "tnpA":1, "dacT": "N", } } }</pre>
UPDATE Wagon Monitoring	UPDATE Wagon Monitoring Sample Payload


```
{
  "op":3,
  "to":"<flexcontainer-ResourceId>",
  "fr":"<DEVICE-AE-ID>",
  "rqi":"<unique-Request-Identifier>",
  "rvi":"3",
  "pc":{
    "hd:wgps": {
      "fw":<String value>,
      "pkSt":<String value>,
      "dgt":<String value>,
      "lat":<String value>,
      "latD":<String value>,
      "lng":<String value>,
      "lngD":<String value>,
      "spd":<Decimal value>,
      "noS":<Decimal value>,
      "hdop":<Decimal value>,
      "pdop":<Decimal value>,
      "alt":<Decimal value>,
      "mcc":<String value>,
      "mnc":<String value>,

```

```
{
  "op":3,
  "to":"R4963",
  "fr":"SVendorCode1",
  "rqi":"Get",
  "rvi":"3",
  "pc":{
    "hd:wgps": {
      "fw":"ABCD1234",
      "pkSt":"L",
      "dgt":"2023-12-12:10:15:34",
      "lat":"00.000000",
      "latD":"N",
      "lng":"00.000000",
      "lngD":"E",
      "spd":100.00,
      "noS":34,
      "hdop":1,
      "pdop":1
      "alt":4000,
      "mcc":"1234",
      "mnc":"1234",

```

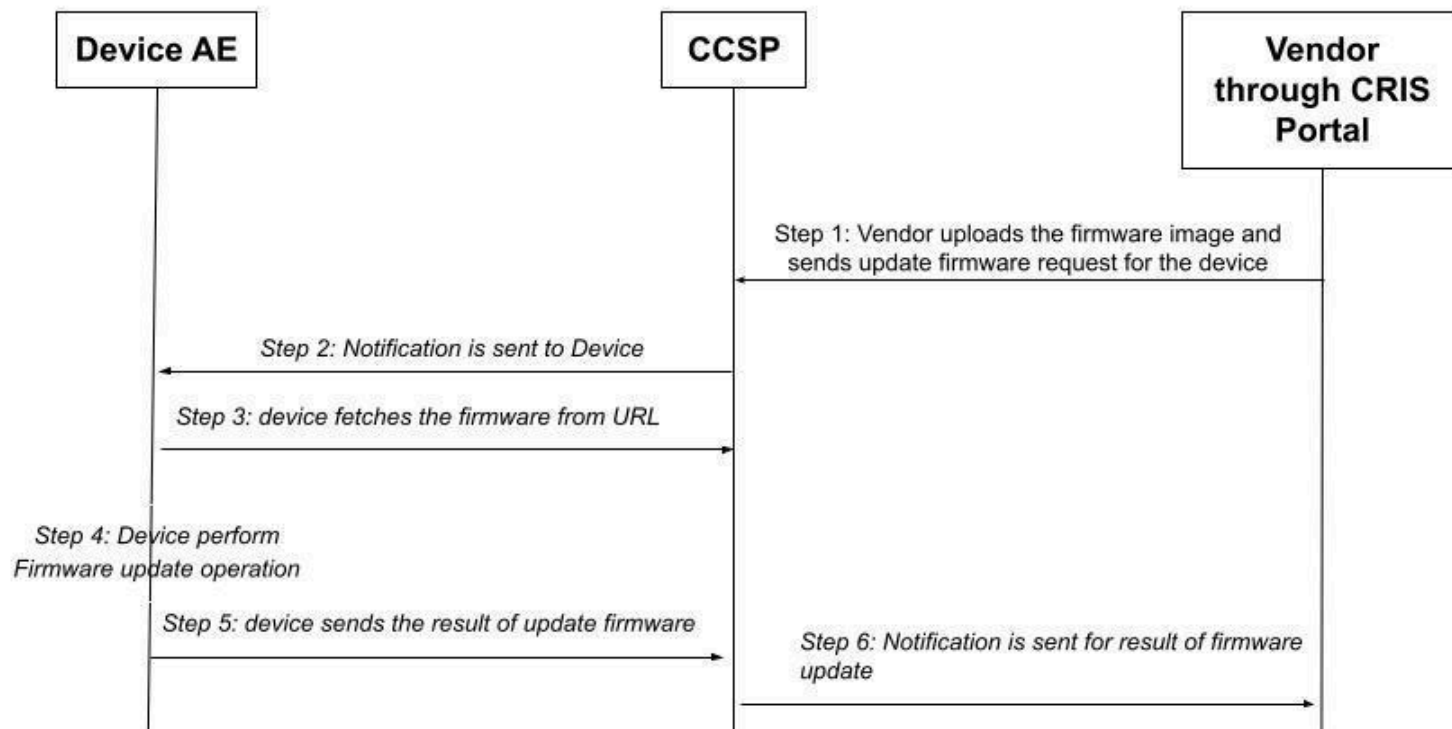
<pre> "lac":<String value>, "cellId":<String value>, "rsi":<Decimal value>, "ber":<Decimal value>, "btPer":<Decimal value>, "mvnt":<Decimal value>, "tmp":<Decimal value>, "dst":<String value>, "cnt":<Decimal value>, "recF":<Decimal value> } } } </pre>	<pre> "lac": "1234", "cellId": "1234", "rsi": 31, "ber": 7, "btPer": 100, "mvnt": 1, "tmp": 55.55, "dst": "2023-12-12:10:15:34", "cnt": 100, "recF": 1 } } } </pre>
Control <pre> { "op": 1, "to": "<CONTAINER-RESOURCE-ID>", "fr": "<DEVICE-AE-ID>", "rqi": "<unique-Request-Identifier>", "rvi": "3", "ty": 4, "pc": { </pre>	Control Sample Payload <pre> { "op": 1, "to": "R4963", "fr": "SVendorCode1", "rqi": "<unique-Request-Identifier>", "rvi": "3", "ty": 4, "pc": { </pre>

<pre>"m2m:cin": { "con": { "ctlTy":<String value>, "ctlVal":<String value>, } } }</pre>	<pre>"m2m:cin": { "con": { "ctlTy": "01", "ctlVal": "1", } } }</pre>
---	--

9.0 Notification Payload: (Sample Notify Message) (Flows from CCSP to Device)

```
{
  "op": 5,
  "to": "<registrarURL>/oneM2M/req/<registrarCseId>/<Device-AE-ID>/json",
  "fr": "/<registrarCseId>",
  "rqi": "<unique-Request-Identifier>",
  "ty": 4,
  "pc": {
    "m2m:sgn": {
      "nev": {
        "rep": {
          "m2m:cin": {
            "con": {
              "ctlTy": "<String value>",
              "ctlVal": "<String value>"
            }
          }
        }
      },
      "net": 3
    },
    "sur": "/CSE001/R2524",
    "cr": "Svendorcode1"
  }
},
  "rvi": "3"
}
```

10.0 Firmware Update procedure



Step 1: Firmware image is uploaded and firmware update request is sent

Step 2: Device receives the notification for firmware update

Device shall process the notification message (mentioned below) for firmware url and version information:

Sample Notification message for firmware update

```
{
  "op" : 5,
  "to" : "mqtt://ccsp.m2m.cdot.in/oneM2M/req/CSE001/S0/json",
  "fr" : "/CSE001",
  "rqi" : "d8d6853f-9ac9-4b55-9c65-b782e450544b",
  "pc" : {
    "m2m:sgn" : {
      "nev" : {
        "rep" : {
          "hd:updFe" : {
            "ty" : 28,
            "ri" : "R144297",
            "pi" : "<dmFirmware-Resource-ID>",
            "ct" : "20230816T095204",
            "lt" : "20230816T095204",
            "rn" : "updFe196013",
            "et" : "20230820T103912",
            "st" : 0,
            "cr" : "<>",
            "cnd" : "org.onem2m.management.action.updateFirmware",
            "cs" : 416,
            "versn" : "2.0",
            "url" : "http://download.com/Beta-Release/2.0"
          }
        },
        "net" : 3
      },
      "sur" : "/CSE001/R144295",
      "cr" : "S0"
    }
  },
  "rvi" : "3"
}
```

Step 3: Device fetches the firmware from url received in notification message in step 2.

Step 4: Device upgrades the firmware with the fetched firmware. In case the device fails to upgrade the firmware on it then rolls back to the previous version.

Step 5: Device sends the result of firmware update to CCSP.

**UPDATE updateFirmware message (to be sent by device) for updating the result
updateFirmware-ResourceID (to be set to “ri” received in notification message in step 2)**

```
{
  "op": 3,
  "to": "<updateFirmware-Resource-ID>",
  "fr": "<Device-AE-ID>",
  "rqi": "<unique-Request-Identifier>",
  "pc": {
    "hd:updFe" : {
      "resut" : true
    }
  },
  "rvi": "3"
}
```

Step 6: Notification is sent for result of firmware update

11.0 Error Messages and Handling

Error codes	Description	Handling required at device
4000	BAD_REQUEST	any mandatory header missed or header data type is invalid
4001	RELEASE_VERSION_NOT_SUPPORTED	RVI (Release version indicator) is set to a value other than 3.
4004	NOT_FOUND	“To” parameter is set to a non-existent resource identifier
4103	ORIGINATOR_HAS_NO_PRIVILEGE	device AE has no privileges to perform operation on the targeted resource.
4105	CONFLICT	same resource name is sent again under the same parent.
4106	ORIGINATOR_NOT_AUTHENTICATED	SubjectAltName set in certificate and From parameter set in request are mismatched.
4126	APP_RULE_VALIDATION_FAILED	APP-ID format is not correct.
4139	SERVICE_SUBSCRIPTION_NOT_ACTIVE	Service subscription is not active and system administrator should be contacted.
4108	INVALID_CHILD_RESOURCE_TYPE	when an invalid resource creation request is sent under a parent.
4117	ORIGINATOR_HAS_ALREADY_REGISTERED	when registration request is received for an already registered device.

ANNEXURE 1**12.0 Parameter Description with Abbreviations Used**

SI No	Key	Key	Data type	Data length	Description
1	fw	Firmware Version	Alphanumeric	10	Firmware Version
2	devId	Device Id	Alphanumeric	10	Unique ID of the Device
3	pkSt	Packet Status	Alphabet	1	L – Live Packet, H – History Packet
4	dgt	Datetime of GPS fix	Date Timestamp		Date and time of location the location obtained from the data
5	lat	Latitude	Floating		Latitude in decimal degrees - dd.dddddd format
6	latD	Latitude Direction	Alphabet	1	N – North, S – South
7	lng	Longitude	Floating		Longitude in decimal degrees - dd.dddddd format
8	lngD	Longitude Direction	Alphabet	1	E – East W – West

9	spd	Speed	Floating	Up to 1 decimal	Speed of Vehicle as Calculated by GPS module. (in km/hr)
10	noS	No. of Satellite	Numeric		
11	hdop	Horizontal Dilution of Precision	Numeric		
12	pdop	Positional Dilution of Precision	Numeric		
13	alt	Altitude	Numeric		Altitude in meters (above sea level)
14	mcc	MCC	Alphanumeric	6	Mobile Country Code
15	mnc	MNC	Alphanumeric	6	Mobile Network Code
16	lac	LAC	Alphanumeric	6	Location Area Code
17	cellId	Cell ID	Alphanumeric	6	Cell ID
18	rsqi	CSQ RSSI	Numeric		The signal strength level value (0-31 99)
19	ber	CSQ BER	Numeric		The quality of the signal. Bit error rate value (0 - 7 99)
20	btPer	Battery Percentage	Numeric		0 - 100
21	mvnt	Movement	Boolean	1	0 1 from accelerometer

		Status			
22	tmp	Temperature	Floating		Temperature of Device in Celcius
23	dst	Data Push Datetime	Date Timestamp		Packet send datetime from device to server
24	cnt	Count Number	Numeric		Count number of packet after reset
25	bleV	BleV	Numeric		Bluetooth vibration value (initially fixed to 0)
26	bleL	BleL	Numeric		Bluetooth load value in ton (initially fixed to 0)
27	bleS	Bluetooth Status	Boolean	1	0 1 Bluetooth on or off
28	tftA	Theft Alert flag	Boolean	1	0 1 Theft Alert on or off
29	tnpA	Tamper Alert flag	Boolean	1	0 1 Tamper Alert on or off
30	dacT	Deactivation Type	Alphanumeric	1	P-Permanent; T-Temporary; S-Strategic; N-None
31	recF	Recovery_flag	Boolean	1	0 1 Recovery mode on or off

ANNEXURE 2

13.0 Pre-defined Values of Parameters

Control Type Detail (Example Control Value):

- 01: Device Reset (1)
- 02: Tracking Periodicity (Running) (24)
- 03: Switching to NavIC (0 in case of Off | 1 in case of On)
- 04: Bluetooth module (0 in case of Off | 1 in case of On)
- 05: Accelerometer Status (0 in case of Off | 1 in case of On)
- 06: Permanent Device deactivation (1)
- 07: Strategic Device deactivation (1)
- 08: Temporary Device deactivation (1)
- 09: Device Activation (1)
- 10: Recovery Mode (0 in case of Off | 1 in case of On)

ANNEXURE 3

14.0 Generic Steps for creating a certificate using OpenSSL

Please note that below steps are described w.r.t Linux OS.

14.1 Generate a CSR (Certificate Signing Request) with SubjectAltName (SAN) extension

Generate CSR using following steps:

- Login into a system where you have OpenSSL installed(Linux)
- Prepare a configuration file which will be used to pass subjectAltName.

sample req.cnf file:

```
ts      = 2048
distinguished_name = req_distinguished_name
req_extensions    = req_ext
[ req_distinguished_name ]
countryName       = Country Name (2 letter code)
stateOrProvinceName = State or Province Name (full name)
localityName      = Locality Name (eg, city)
organizationName  = Organization Name (eg, company)
commonName        = Common Name (e.g. server FQDN or YOUR name)
[ req_ext ]
subjectAltName = @alt_names
[alt_names]
URI.1=<Add SAN After removing 'URI:'>
```

e.g. URI.1= <https://ccsp.m2m.cdote.in/SP1>

Execute below OpenSSL command:

```
openssl req -out client.csr -newkey rsa:2048 -nodes -keyout private.key -config req.cnf
```

This will create [client.csr](#) and [private.key](#) in the present working directory. You have to send [client.csr](#) to certificate signer authority so they can provide you a certificate with SAN.

14.2 Generic Steps for creating a certificate using EasyRSA

Please note that below steps are described w.r.t Linux OS.

14.3 Generate a CSR (Certificate Signing Request) with SubjectAltName (SAN) extension

Generate CSR using following steps:

- Login into a server where you have OpenSSL installed (Linux)
- Download the easyRSA directory from following link in some new directory and perform steps mentioned below :
 1. `wget https://github.com/OpenVPN/easy-rsa/releases/download/v3.1.2/EasyRSA-3.1.2.tgz`
 2. `tar -xf EasyRSA-3.1.2.tgz`
 3. `cd EasyRSA-3.1.2`
 4. `./easyrsa init-pki`
 5. `./easyrsa --subject-alt-name='subjectAltName' gen-req $certificate_name nopass`
 - a. **Example:** `./easyrsa --subject-alt-name='URI:https://ccsp.m2m.cdote.in/SP1' gen-req client1 nopass`

This will create `client1.req` in `EasyRSA-3.1.2/pki/reqs` and `client1.key` in `EasyRSA-3.1.2/pki/private` the present working directory. You have to send `client1.req` to certificate signer authority so they can provide you a certificate with SAN.

14.4 Getting CSR signed from the CDOT-M2M-CA

Step 1: Download CDOT CCSP CA public key certificate (ca.crt file) (Sign in to CCSP Portal → Go to SSM Module → Click on Certificate Management → Download root CA → click on download button to download root CA certificate)

Step 2: Go to the directory where the generated csr/req file is present.

Step 3: `curl -o <Certificate_Name>.crt -F csrFile=@<CSR>.req --cacert $CERT_HOME/ca.crt https://<CADomainName>/getCert`

or

```
curl -o <Certificate_Name>.cert -F csrFile=@<CSR>.csr --cacert $CERT_HOME/ca.crt  
https://<CADomainName>/getCert
```

Note: CERT_HOME is the path where the certificate is downloaded in step 1.

Description of above command:

<Certificate_Name> : name of the output certificate file received from CA

<CSR> : name of the csr/req file to be sent to CA

--cacert : flag to set the path of ca certificate for https connection to CA server

ca.crt : Actual path of ca certificate

<CADomainName> : URL of CA server e.g. <https://ca.m2m.cdote.in>

<cert_type> : server for CSE, client for AE

Example:

```
curl -o client1.cert -F csrFile=@client1.req --cacert ca.crt https://<CADomainName>/getCert
```

ANNEXURE 4

15.0 Variables used in the JSON payloads

Variable Name	Details
<DEVICE-AE-ID>	Represents AE-ID for any field device
<CRIS-AE-ID>	Represents AE-ID for CRIS Portal AE

<registrarCseld>	Represents CSE-ID for the CCSP
<registrarUrl>	Represents the Broker URL for CCSP e.g. mqtts://ccsp.m2m.cdota.in
<container-ResourceId>	Represents the resourceId for the Container
<flexcontainer-ResourceId>	Represents the resourceId for the flex Container holding the monitoring data of a wagon
<unique-Request-Identifier>	Represents any Unique String Identifier
<CADomainName>	Represents domain name for Certification Authority (CA) e.g. ca.m2m.cdota.in
op	Represents oneM2M operation op=1 CREATE, op=2 RETRIEVE, op=3 UPDATE, op=4 DELETE, op=5 NOTIFY

Note: For other short names used in payload, please refer: <https://coi.cdota.in/docs/oneM2MGuideline.html>

ANNEXURE 5**16.0 Registration of Vendors through Application****16.1 App-ID Registration by vendors**

Vendor will register App-ID '*Ra1.<vendor_domain>.wgps*' at oneM2M App-ID Registry
(<https://appid.atis.org/>)

16.2 Information required from vendor to Railways:

1. Name
2. GSTIN
3. PAN
4. App-ID
5. Address
6. Contact Details:
 - Name
 - Email
 - Number
7. Helpdesk Details:
 - Website
 - Number

Particulars Of Vendor			
Vendor Name *		GSTIN *	
PAN Number *		App-ID *	 Generated from oneM2M App-ID Registry (https://appid.atis.org)
Address (Headquarters) *		☐ Copy Headquarters Address	
Contact Name *		Address (Local) *	
Contact Email *	example@abc.com	Contact Number (Mobile) *	
Website / Portal URL	https://www.example	Helpdesk (Landline)	
		Helpdesk (Tollfree)	

17.0 Contact Details

For understanding oneM2M standards, Container creation, Subscription creation:	For understanding flex container creation, updation and their respective parameters (Annexure 1, Annexure 2):
Name: Poornima Shandilya Email: poornima@cdot.in Mb: 9953110247	Name: Sunil Kumar Email: kumar2.sunil@cris.org.in Mb: 8130693976
Name: Siddharth Trikha Email: strikha@cdot.in Mb: 9958954669	Name: Aman Yadav Email: yadav.aman@cris.org.in Mb: 9971986413